King Fahd Causeway Authority

Technical Assessment Submission

Accident Detection from CCTV Footage Using ResNet18

Technical Write-Up

Prepared by:	Athar Parvez
Role:	Data Science Candidate

Project Type:	Binary Classification of Accident vs Non-Accident CCTV Frames
Model Used:	ResNet18 with Transfer Learning
Submission Date:	March 27, 2026

This document summarizes the preprocessing decisions, model architecture, training strategy, evaluation results, and a production deployment approach on Google Cloud Platform using Vertex AI.

1. Problem Overview

The objective of this project is to build a binary image classification model that can determine whether a CCTV frame contains an accident or a non-accident scene. The dataset is provided in a folder-based image format with separate `train` and `test` directories, making the task well suited for a supervised deep learning pipeline.

From a practical standpoint, this problem is important because accident detection from surveillance footage can support faster incident response, improve traffic monitoring, and serve as a foundation for real-time safety systems. The challenge, however, is that CCTV frames often contain variations in camera angle, lighting, motion blur, background clutter, and scene composition. Because of this, the model needs to learn features that are robust enough to generalize beyond the training samples.

To address this, I approached the task as a transfer learning problem using ResNet18, a lightweight and well-established convolutional neural network architecture pretrained on ImageNet. This choice provides a strong balance between representational power, training efficiency, and practical deployability.

2. Data Preprocessing Decisions

I started by understanding how the dataset is structured. The data is organized into two main folders, `train` and `test`, and each of them contains two classes: *Accident* and *Non-Accident*. The training set has 371 accident images and 422 non-accident images, while the test set contains 49 accident images and 69 non-accident images. Overall, the dataset is fairly balanced, so I did not need to apply any special techniques for class imbalance.

Before jumping into modeling, I did a quick round of basic exploration. I looked at a few sample images from both classes to get a sense of the data. One thing that stood out is that these are real CCTV-like frames, so there is a lot of variation in lighting, camera angles, and background scenes. In some cases, accidents are not visually obvious, which makes the problem more challenging and reinforces the need for a model that can generalize well.

Since the dataset only provides train and test splits, I created a validation set from the training data. This helps in monitoring model performance during training without touching the test set, which I kept unseen for final inference.

For preprocessing, I resized all images to 224×224 to match the input requirements of ResNet18. During training, I applied simple augmentations such as horizontal flips and slight rotations to make the model more robust to variation in real-world data. For validation and test data, I only applied resizing and normalization, without augmentation, so that evaluation remains consistent and realistic.

Overall, the preprocessing step was kept simple but effective, with the main goal of improving generalization while keeping the pipeline clean and reproducible.

3. Model Architecture Choice (ResNet18)

For this task, I used ResNet18 with a transfer learning approach. The main reason for choosing ResNet18 is that it offers a good balance between performance and computational efficiency, which makes it suitable for a relatively small dataset like this one.

Instead of training the model from scratch, I initialized it with pretrained ImageNet weights. This allows the model to leverage already learned low-level and mid-level visual features such as edges, shapes, and textures, which helps with faster convergence and better generalization.

To adapt the model to this problem, I followed a partial fine-tuning strategy. I first froze all layers to retain the pretrained knowledge and then selectively unfroze the last block (`layer4`) so that the model could learn more task-specific features relevant to accident detection. This approach strikes a useful balance between stability and flexibility during training.

I also replaced the final fully connected layer with a custom classification head consisting of a dropout layer followed by a linear layer. The dropout rate of 0.3 helps reduce overfitting, while the final layer maps the extracted features to the two target classes: Accident and Non-Accident.

Overall, this setup allows the model to reuse strong pretrained features while still adapting effectively to the specific characteristics of CCTV accident data.

4. Training Approach

For training, I kept the setup simple but structured to ensure stable learning and good generalization.

I used `CrossEntropyLoss` as the loss function, which is a standard choice for classification tasks. For optimization, I used the Adam optimizer and updated only the trainable parameters, namely the unfrozen final ResNet block and the custom classification head. This allowed the model to preserve most of the pretrained knowledge while still adapting to the accident detection task.

To manage learning more effectively during training, I used a `ReduceLROnPlateau` scheduler based on validation loss. The training started with a learning rate of `0.0001`, and when validation performance stopped improving consistently, the scheduler reduced it to `0.00005`. This helped stabilize the optimization process in the later stages of training.

The training pipeline was implemented with separate training and validation steps for each epoch. During training, the model performed forward and backward propagation and updated the weights. During validation, the model was evaluated without gradient updates, and I tracked loss, accuracy, and weighted F1-score.

From the training logs, the model improved steadily during the first several epochs. Validation F1-score increased from approximately `0.7529` in the first epoch to its best value of `0.9117`. The strongest validation performance was reached around epoch 11, after which the gains became marginal. Since the model stopped showing meaningful improvement, early stopping was triggered at epoch 18, even though the maximum training budget was set to 50 epochs.

Instead of selecting the model based only on accuracy, I used validation F1-score as the main checkpointing metric. This provided a better balance between precision and recall, which is more appropriate for an accident detection problem where both missed detections and false alarms matter.

Overall, the training strategy worked well in practice. The model learned quickly in the early epochs, the scheduler helped refine training after performance plateaued, and early stopping prevented unnecessary training once the model had already converged.

5. Evaluation Results

After training, the model was evaluated on the validation set using accuracy, precision, recall, and F1-score.

The model achieved an overall accuracy of 91.19, with precision, recall, and F1-score all close to 0.91. This indicates that the model performs consistently across both classes without any strong bias.

At the class level, the model shows slightly different behavior. For the Accident class, precision is around 0.93 while recall is around 0.88, which means some accident cases are missed. For the Non-Accident class, recall is higher at around 0.94, showing that normal scenes are detected more reliably.

The confusion matrix shows that 87.84% of accident cases and 94.12% of non-accident cases are correctly classified. Overall, the number of misclassifications is low and reasonably balanced.

From the training curves, the model improves steadily in the early epochs. The learning rate is reduced when validation performance stops improving, and early stopping prevents unnecessary training. There is no significant overfitting, and validation performance remains stable.

Figures below show the training behavior and evaluation results.

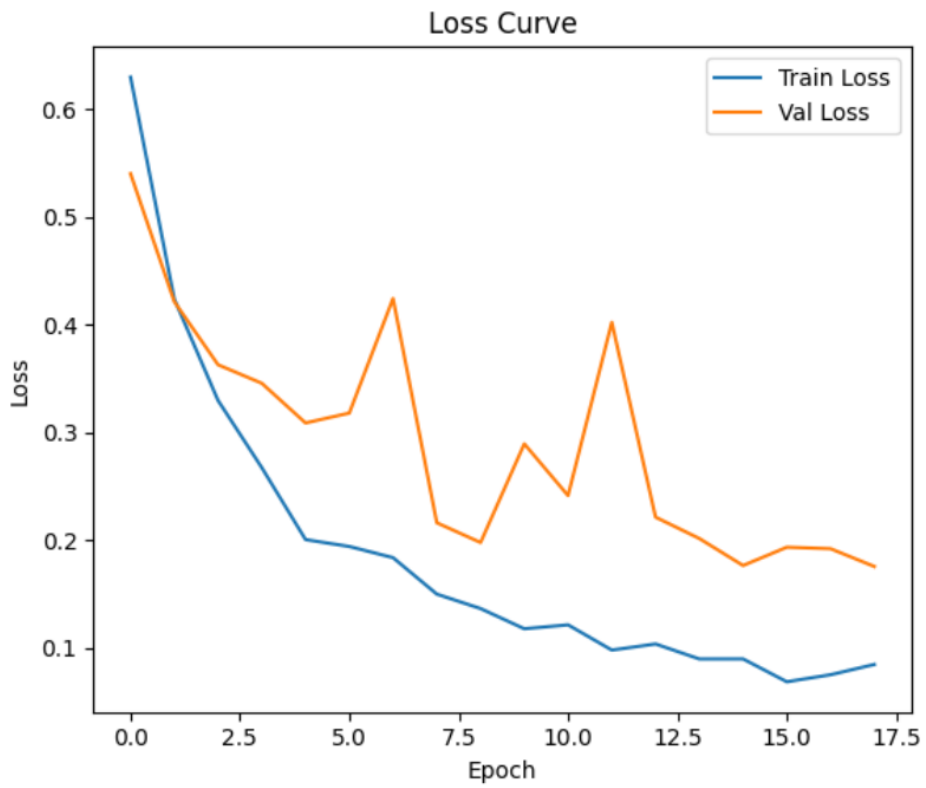

Figure 1: Training and validation loss over epochs

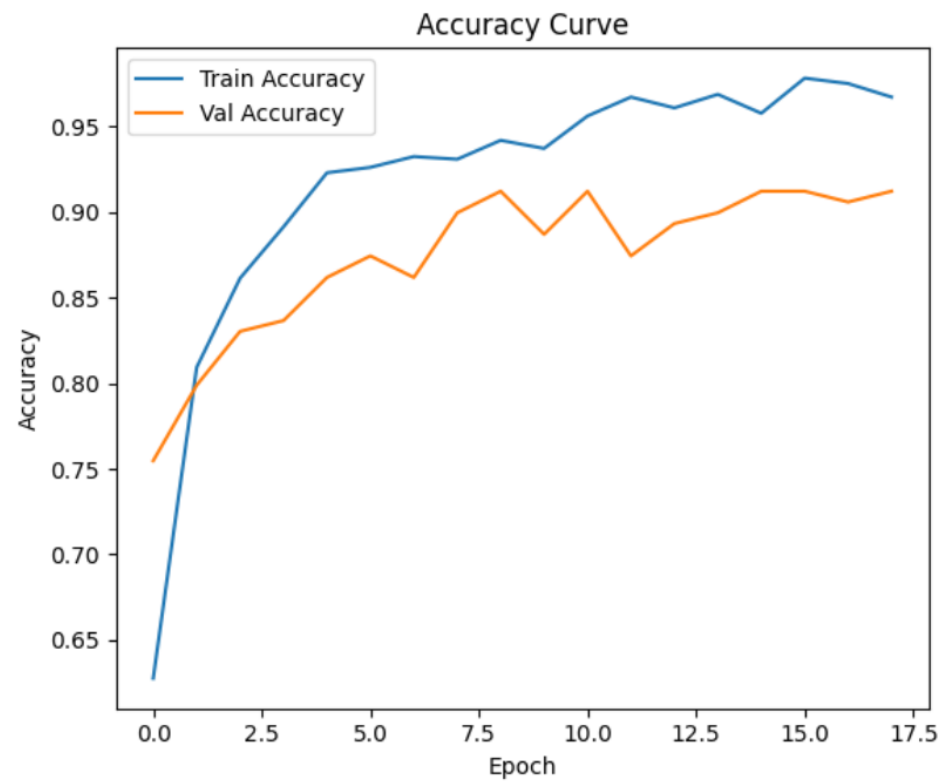

Figure 2: Training and validation accuracy over epochs

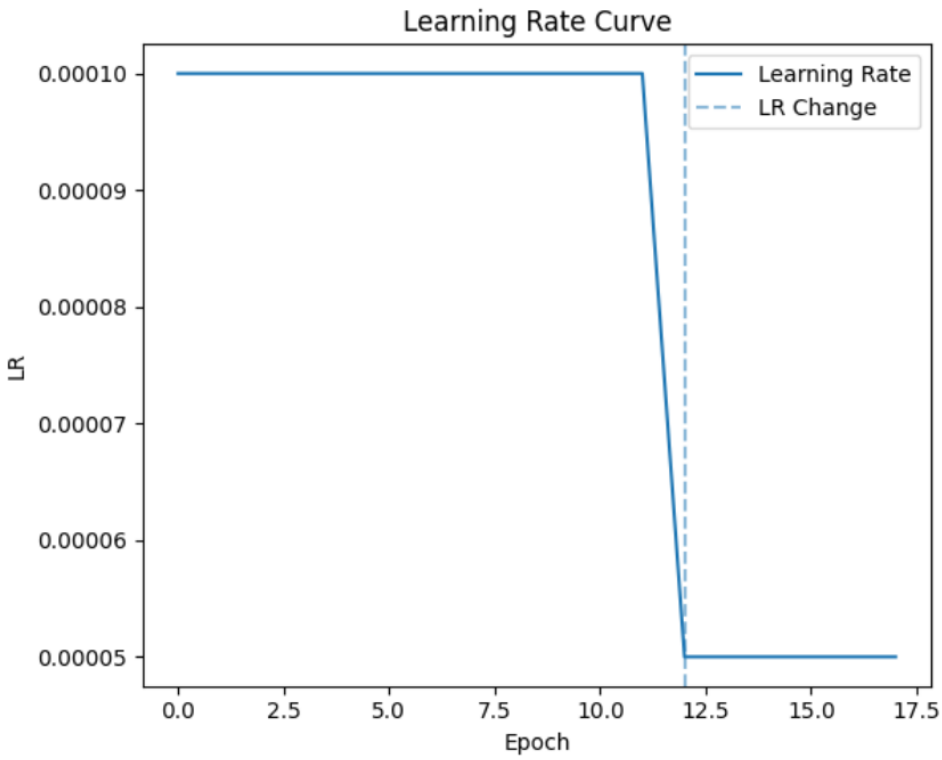

Figure 3: Learning rate schedule during training

```
--------------------------------------------------
Accuracy : 0.9119
Precision: 0.9127
Recall   : 0.9119
F1 Score : 0.9117

--------------------------------------------------
Classification Report:
              precision    recall  f1-score   support

    Accident       0.93      0.88      0.90        74
Non Accident       0.90      0.94      0.92        85

    accuracy                           0.91       159
   macro avg       0.91      0.91      0.91       159
weighted avg       0.91      0.91      0.91       159
```

Figure 4: Classification report on validation set

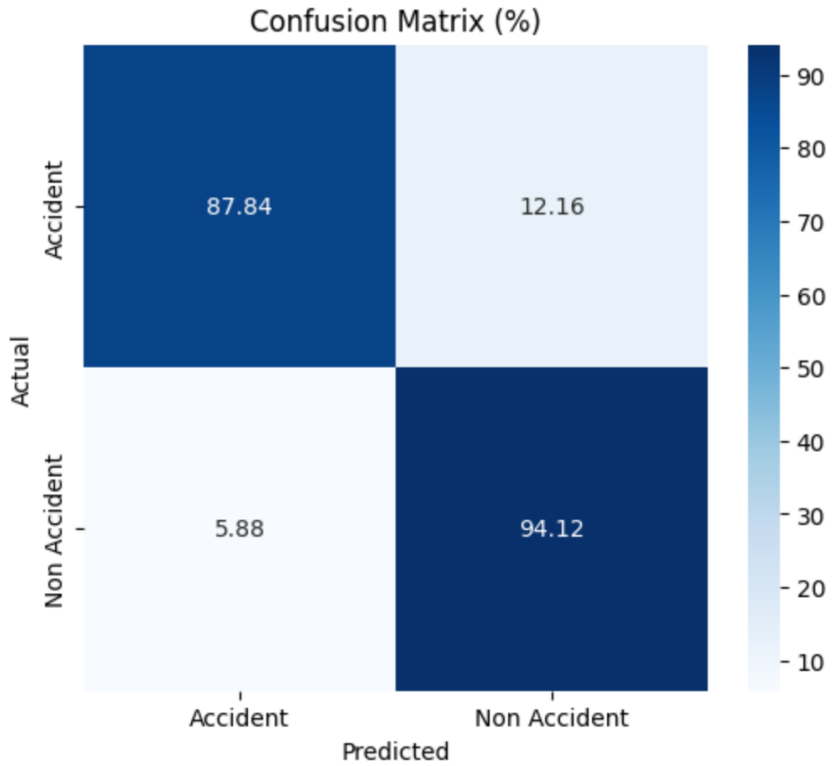

Figure 5: Confusion matrix (percentage)

6. Deployment on GCP (Vertex AI Approach)

If I were to deploy this model in a production environment, I would use Google Cloud Platform (GCP) with Vertex AI for model serving and lifecycle management.

During training, I saved the best-performing model as a `.pth` file based on validation F1-score. For deployment, this model could be used directly within a custom serving setup or optionally converted to a format such as TorchScript or ONNX for better portability and serving efficiency.

For serving, I would deploy the model using Vertex AI Endpoints. This would allow the model to be exposed as a scalable API capable of handling real-time prediction requests. The endpoint could automatically scale based on traffic, which is useful in a production setting.

In a real-world CCTV pipeline, frames would typically be captured continuously and stored in Cloud Storage. Using services such as Pub/Sub or Cloud Functions, an inference trigger could be configured so that each new frame is preprocessed and sent to the Vertex AI endpoint. The model would then return predictions along with confidence scores.

If an accident were detected with high confidence, the system could trigger downstream alerts or notifications. In parallel, prediction outputs and metadata could be stored in BigQuery for monitoring, analytics, and future retraining.

To ensure long-term reliability, I would monitor prediction trends and data drift over time. If the data distribution changed or model performance degraded, the model could be retrained using newly collected data and redeployed through the same pipeline.

Overall, Vertex AI provides a scalable and maintainable path for deploying and managing this model in a real-world environment.

7. Conclusion

This project focused on building a practical binary image classification pipeline for detecting accidents from CCTV footage. A transfer learning approach with ResNet18 was chosen to balance model quality, training efficiency, and deployability. The final model delivered strong validation performance while keeping the overall workflow clean and reproducible.

From a broader perspective, the solution is not only suitable for a technical test setting but also provides a reasonable foundation for a production deployment on GCP using Vertex AI. With additional data, monitoring, and periodic retraining, this approach could be extended into a more robust real-world accident detection system.